**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**CSA0607 Design and Analysis of Algorithms**

# ASSESSMENT– CO3

Brute-Force and Exhaustive-Search Techniques

## Complete Assessment Report

| Item | Details |
|---|---|
| Student name | RITHIKA G |
| Register number / Section | 192571027 |
| Course Outcome | **CO3**- Module 1 – Nightly Order Consolidation (Merge Sort) |
| Nature / Marks | Individual · 50 marks |
| GitHub repository | https://github.com/192571027/CSA0607- |
| Implementation language | Python 3.12 (matplotlib for graphs) |

# Module 1 – Nightly Order Consolidation (Merge Sort)

**Case:** KartIQ records about 10,000,000 orders a day. All orders must be sorted by timestamp for the audit report, and orders with the same timestamp must keep their original (gateway arrival) order because auditors reconcile gateway by gateway.

**Chosen algorithm:** Stable top-down Merge Sort (external k-way merge when data exceeds RAM).

---

## 1. Divide, Conquer and Combine

| Step | What Merge Sort does | In the code |
|---|---|---|
| **Divide** | Split the range A[lo..hi) at mid = ⌊(lo+hi)/2⌋ into two halves of size ⌈n/2⌉ and ⌊n/2⌋. Costs Θ(1). | mid = (lo + hi) // 2 |
| **Conquer** | Recursively sort both halves. A range of 0 or 1 elements is already sorted (base case). | two recursive calls |
| **Combine** | Merge the two sorted halves by repeatedly taking the smaller front element. Costs at most n − 1 comparisons for n elements. On a tie the **left** element is taken first. | merge() using <= |

The key to stability is the single character <= in the merge: when aux[i].timestamp == aux[j].timestamp, the element from the left run (which arrived earlier) is copied first.

---

## 2. Trace on the sample orders

Input in arrival order: (105,O1) (102,O2) (105,O3) (101,O4) (102,O5) (103,O6)

### Division tree

[(105,O1) (102,O2) (105,O3) (101,O4) (102,O5) (103,O6)]
├── [(105,O1) (102,O2) (105,O3)]
│ ├── [(105,O1)]
│ └── [(102,O2) (105,O3)]
│ ├── [(102,O2)]
│ └── [(105,O3)]
└── [(101,O4) (102,O5) (103,O6)]
 ├── [(101,O4)]
 └── [(102,O5) (103,O6)]

├── [(102,O5)]
└── [(103,O6)]

### Every merge (bottom-up, as executed)

| # | Left run | Right run | Comparisons made (tie → left) | Result | Cmp |
|---|---|---|---|---|---|
| 1 | (102,O2) | (105,O3) | 102 ≤ 105 → take O2 | (102,O2) (105,O3) | 1 |
| 2 | (105,O1) | (102,O2) (105,O3) | 105 > 102 → take **O2**; 105 ≤ 105 (tie) → take **O1** (left) | (102,O2) (105,O1) (105,O3) | 2 |
| 3 | (102,O5) | (103,O6) | 102 ≤ 103 → take O5 | (102,O5) (103,O6) | 1 |
| 4 | (101,O4) | (102,O5) (103,O6) | 101 ≤ 102 → take O4; left exhausted, copy rest | (101,O4) (102,O5) (103,O6) | 1 |
| 5 | (102,O2) (105,O1) (105,O3) | (101,O4) (102,O5) (103,O6) | 102 > 101 → O4; 102 ≤ 102 (tie) → take **O2** (left); 105 > 102 → O5; 105 > 103 → O6; right exhausted, copy O1, O3 | **(101,O4) (102,O2) (102,O5) (103,O6) (105,O1) (105,O3)** | 4 |

Total comparisons: 1 + 2 + 1 + 1 + 4 = **9** (the worst-case bound for n = 6 is 11).

### Output verification – stability

| Timestamp | Arrival order in input | Order in sorted output | Preserved? |
|---|---|---|---|
| 102 | O2, O5 | O2, O5 | ✓ |
| 105 | O1, O3 | O1, O3 | ✓ |

The two decisive moments are merge 2 (tie 105/105, O1 from the left run wins over O3) and merge 5 (tie 102/102, O2 from the left run wins over O5). The program also verifies programmatically that the output is sorted and that for every pair of equal timestamps the arrival index increases (is_stable_and_sorted → True).

**Why an unstable sort would fail the audit.** Quick Sort's partition swaps can jump an element over equal ones. Example: (5,a) (5,b) (5,c) (1,d) with a first-element-pivot Lomuto partition gives (1,d) (5,a) (5,c) (5,b) – orders b and c from the same second have changed places, so the gateway-by-gateway reconciliation would break. (Alternatively an unstable sort could be made stable by sorting on the pair *(timestamp, arrival index)*, but that needs an extra key per record and does not use the guarantee that Merge Sort gives for free.)

---

## 3. Recurrence and solution

Let T(n) be the number of key comparisons in the worst case.

$T(1) = 0$

$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + (n - 1) \qquad \text{for } n \geq 2$

For $n = 2^k$ this is $T(n) = 2T(n/2) + n - 1$.

**Recursion-tree solution.** At level $i$ there are $2^i$ sub-problems of size $n/2^i$, and each merge costs $(n/2^i - 1)$, so level $i$ costs $n - 2^i$. Summing over the $\lg n$ levels ($i = 0 \ldots \lg n - 1$):

$T(n) = \Sigma\,(n - 2^i) = n \cdot \lg n - (2^{\lg n} - 1) = n \cdot \lg n - n + 1$

**Master theorem check.** $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$ ⇒ case 2 ⇒ **$T(n) = \Theta(n \log n)$**.

**General n.** $T(n) = n\lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$.

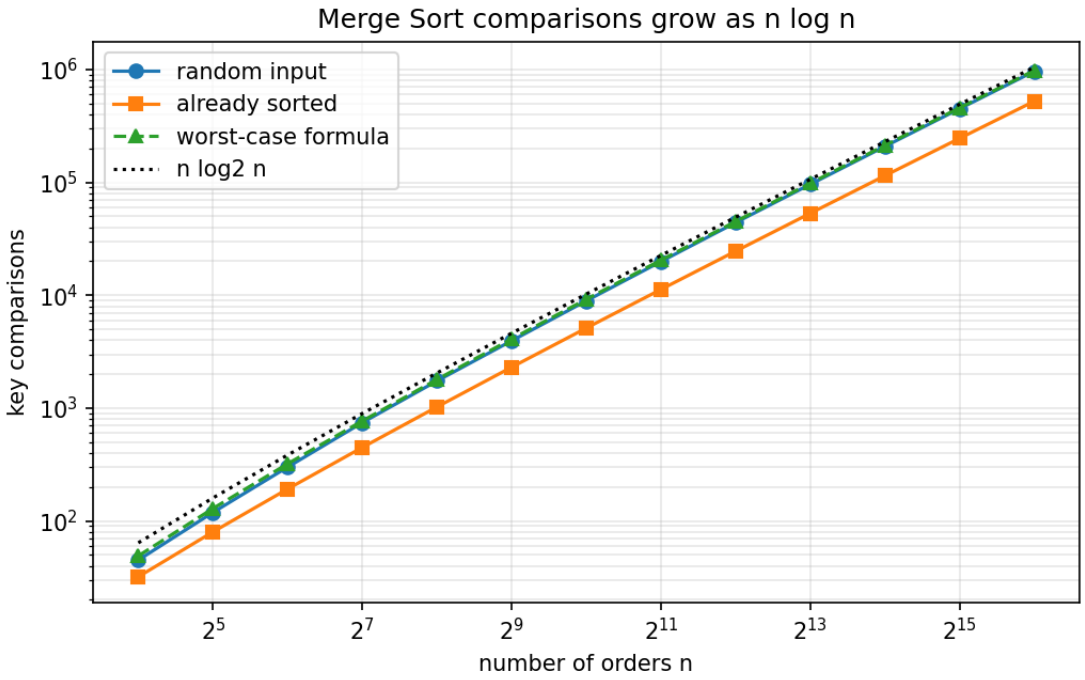
| Case | Comparisons | Order |
|---|---|---|
| Worst | $n\lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$ | $\Theta(n \log n)$ |
| Best (already sorted: every merge ends when the left run is used up) | $\approx (n/2) \cdot \lg n$ | $\Theta(n \log n)$ |
| Average (random) | $\approx n \lg n - 1.25n$ | $\Theta(n \log n)$ |

Merge Sort is not input-sensitive in its order of growth: it is $\Theta(n \log n)$ in all cases, which is what a nightly job with a deadline needs.

## Experimental confirmation (from merge_sort.py)

| n | random | sorted | reverse | worst-case formula | n lg n − n + 1 |
|---|---|---|---|---|---|
| 1,024 | 8,921 | 5,120 | 5,344 | 9,217 | 9,217 |
| 8,192 | 96,103 | 53,248 | 54,980 | 98,305 | 98,305 |
| 65,536 | 965,552 | 524,288 | 538,095 | 983,041 | 983,041 |

For $n$ a power of two, the worst-case formula equals $n \lg n - n + 1$ exactly, and the measured counts for random data stay just below it.




*Merge sort comparisons*

---

## 4. Estimate for 10,000,000 orders

| Quantity | Calculation | Value |
|---|---|---|
| lg n | $\log_2(10^7)$ | 23.2535 ($\lceil\cdot\rceil = 24$) |
| n lg n | $10^7 \times 23.2535$ | 232,534,967 |
| Worst-case comparisons | $10^7 \times 24 - 2^{24} + 1 = 240{,}000{,}000 - 16{,}777{,}216 + 1$ | **223,222,785 ≈ 2.2 × $10^8$** |
| Approximation n lg n − n + 1 | 232,534,967 − 10,000,000 + 1 | 222,534,968 |
| Best case | (n/2) lg n | ≈ $1.16 \times 10^8$ |
| Recursion depth | ⌈lg n⌉ | 24 levels |

About $2.2 \times 10^8$ comparisons is a few seconds of CPU work in a compiled language, so the algorithm itself is not the bottleneck – memory and I/O are.

---

## 5. Memory requirement and handling data larger than RAM

**In-memory requirement.** Merge Sort needs an auxiliary buffer: Θ(n) extra space (the recursion stack adds only Θ(log n)).

| Item | Calculation (16 B per record: 8 B timestamp + 8 B order ID) | Size |
|---|---|---|
| Data array | 10,000,000 × 16 B | 160 MB |
| Full auxiliary buffer | same size | 160 MB (total 320 MB) |
| Half-size buffer (copy only the left run) | n/2 × 16 B | 80 MB (total 240 MB) |

Reductions: a half-size buffer, ping-ponging between two arrays instead of copying back at every merge, and sorting an array of 4-byte indices/keys instead of whole records when records are large.

**When the data does not fit in RAM – external merge sort** (implemented and verified in external_merge_sort):

1. **Run generation.** Read M records that fit in memory (e.g. M = 1,000,000 ≈ 16 MB, or much larger in practice), sort them with the stable in-memory Merge Sort, and write the run to disk. Runs are numbered in arrival order. 10 M orders ⇒ 10 runs.
2. **k-way merge.** Open all k runs with one small input buffer each and merge with a min-heap keyed on (timestamp, run number). Ties are broken by run number, i.e. by arrival order, so **stability is preserved across runs**. Cost: n·lg k comparisons and one sequential read and write of the whole file.

Totals stay Θ(n log n) comparisons; I/O is about two sequential passes over the data (≈ 4 × 160 MB moved for 10 M orders), and disk access is sequential, which is what disks and SSDs do best. If k is larger than the number of buffers that fit in memory, merge in several passes (⌈log_f k⌉ passes for fan-in f). The demo sorts 20,000 orders in 7 runs of ≤ 3,000 and the result is identical to the in-memory stable sort.

---

## 6. Why Merge Sort is the right choice here

- **Stability** is a hard requirement – Merge Sort has it, Quick Sort and Heap Sort do not.
- **Guaranteed Θ(n log n)** in every case, so the nightly job has a predictable run time.
- **Sequential access pattern** – suits external sorting and parallel execution (the two halves are independent).
- **Cost:** Θ(n) extra memory; this is the trade-off, mitigated as shown above.